

Programmieren für Kids

JavaScript für Kids

Modul 8

Klassiker-Spiele



Trainer: Wolfgang Schedl

Kurz zurückgeblickt

Du hast schon eine Menge gebaut. Jetzt bringst du Spiele auf den Bildschirm, die **jeder** kennt. Dafür brauchst du nur ein paar neue **Helfer**, kleine Werkzeuge, die dir Arbeit abnehmen. Dann geht es an die Klassiker.



Das baust du in diesem Modul

- **Galgenmännchen:** rate das Wort, Buchstabe für Buchstabe
- **Drei gewinnt:** Tic-Tac-Toe für zwei Spieler
- **Memory:** finde die Paare

Spiele, die deine Freunde sofort erkennen.



Kapitel 1

Neue Helfer

← return: etwas zurückgeben

Bisher haben Funktionen etwas *getan*. Eine Funktion kann dir aber auch ein Ergebnis **zurückgeben**:

```
function zufallsZahl() {  
  return Math.floor(Math.random() * 13) + 1  
}  
  
let karte = zufallsZahl() // karte ist jetzt eine  
                        Zufallszahl
```

return spuckt einen Wert aus, den du weiterverwenden kannst, wie eine kleine Maschine.

+ Zu einer Liste hinzufügen: push

Mit `.push` hängst du etwas hinten an eine Liste an:

```
let geraten = []  
  
geraten.push("A")    // ["A"]  
geraten.push("B")    // ["A", "B"]
```

Die Liste startet leer und wächst mit jedem `push`.

🔍 Ist es dabei? includes

Mit `.includes` fragst du, ob etwas in einer Liste (oder in einem Wort) steckt:

```
let geraten = ["A", "B"]  
  
geraten.includes("A")    // true  
geraten.includes("Z")    // false
```

Praktisch, um zu prüfen: „Habe ich diesen Buchstaben schon geraten?“

Listen in Listen

Eine Liste kann selbst Listen enthalten, praktisch für Zusammengehöriges:

```
let linien = [[0, 1, 2], [3, 4, 5]]  
  
linien[0]           // [0, 1, 2]  
linien[0][1]        // 1
```

`linien[0]` ist die erste kleine Liste, `linien[0][1]` das zweite Ding darin.

A

Projekt

Galgenmännchen



Jetzt öffnen: 8a-galgenmaennchen

in VS Code – oder auf der Kursseite

A Wie es geht

- der Computer wählt ein geheimes Wort
- du klickst Buchstaben, einen nach dem anderen
- richtige Buchstaben erscheinen, falsche zählen als Fehler
- bei 6 Fehlern hast du verloren

Neu dabei: `.push` (geratene Buchstaben merken) und `.includes` (steckt er im Wort?).



So ist deine Vorlage aufgebaut

```
<h2 id="wort"></h2>
<p id="fehler">Fehler: 0 / 6</p>
<p id="text">Rate das Wort!</p>
<button onclick="rate('A')">A</button>    <!-- ... B bis Z -->
```

Ein Feld fürs **wort**, eine Fehler-Anzeige und 26 Buchstaben-Knöpfe, die `rate(...)` rufen.

Das Wort anzeigen

Für jeden Buchstaben im Wort: schon geraten, dann zeigen, sonst ein Strich.

```
function zeigeWort() {  
  let anzeige = ""  
  for (let i = 0; i < wort.length; i++) {  
    if (geraten.includes(wort[i])) {  
      anzeige = anzeige + wort[i] + " "  
    } else {  
      anzeige = anzeige + "_ "  
    }  
  }  
  document.getElementById("wort").innerHTML = anzeige  
}
```

Einen Buchstaben raten

```
function rate(buchstabe) {  
  geraten.push(buchstabe)  
  if (wort.includes(buchstabe) === false) {  
    fehler = fehler + 1  
  }  
  document.getElementById("fehler").innerHTML = "Fehler: " +  
    fehler + " / 6"  
  zeigeWort()  
}
```

Buchstabe merken (push), und wenn er *nicht* im Wort steckt, ein Fehler mehr.

Übung: dein Galgenmännchen



PROJEKT

ca. 15 Minuten

- Bring das Spiel zum Laufen (anzeigen und raten)
- Schreibe deine eigenen Wörter in die Liste (GROSS, ohne Umlaute)

★ **Bonus:** Sag „Gewonnen!“, wenn kein Strich mehr übrig ist, und „Verloren!“ bei 6 Fehlern.



Projekt

Drei gewinnt



Jetzt öffnen: 8b-tic-tac-toe

in VS Code – oder auf der Kursseite

■ ■ ■ Wie es geht

- ein Spielfeld mit neun Feldern
- zwei Spieler, X und O, sind abwechselnd dran
- wer drei in einer Reihe hat, gewinnt

Neu dabei: eine **Liste in Listen** für die Gewinn-Linien und `return` für die Prüfung.



So ist deine Vorlage aufgebaut

```
<p id="text">X ist dran</p>  
<button id="z0" onclick="setze(0)"></button>  
<button id="z1" onclick="setze(1)"></button>  <!-- ... bis z8 -->
```

Neun Felder z0 bis z8, jedes ruft `setze(...)` mit seiner Nummer.

Setzen und wechseln

```
let feld = ["", "", "", "", "", "", "", "", ""]
let spieler = "X"

function setze(i) {
  if (feld[i] !== "") return // Feld schon belegt
  feld[i] = spieler
  document.getElementById("z" + i).innerHTML = spieler

  if (spieler === "X") spieler = "O"
  else spieler = "X"
}
```

return bricht früh ab, wenn das Feld schon voll ist. Danach ist der andere dran.

Wer hat gewonnen?

```
function gewonnen() {  
  let linien = [[0,1,2],[3,4,5],[6,7,8],[0,3,6],  
    [1,4,7],[2,5,8],[0,4,8],[2,4,6]]  
  for (let i = 0; i < linien.length; i++) {  
    let a = linien[i][0], b = linien[i][1], c = linien[i][2]  
    if (feld[a] !== "" && feld[a] === feld[b] && feld[b] ===  
      feld[c]) {  
      return true  
    }  
  }  
  return false  
}
```

Übung: dein Drei gewinnt



PROJEKT

ca. 15 Minuten

- Bring das Spiel zum Laufen und spielt zu zweit
- Ruf gewonnen() nach jedem Zug auf und zeig den Sieger

★ **Bonus:** Sag „Unentschieden“, wenn alle Felder voll sind und niemand gewonnen hat.



Projekt

Memory



Jetzt öffnen: [8c-memory](#)

in VS Code – oder auf der Kursseite



Wie es geht

- lauter verdeckte Karten, immer zwei gleiche gehören zusammen
- du deckst zwei auf: passen sie, bleiben sie offen
- passen sie nicht, klappen sie nach kurzer Zeit wieder zu

Neu dabei: ein `setTimeout` aus Modul 6, das die Karten zurückklappt.



Vorlage und Karten

```
// die Karten (immer paarweise!) und einmal mischen:  
let symbole = ["A","A", "...", "...", "...", "...",  
              "...", "..."] // deine, immer paarweise  
symbole.sort(function() { return Math.random() - 0.5 })
```

Acht Knöpfe k0 bis k7 zeigen erst „?“. Das Mischen bekommst du geschenkt.

Eine Karte aufdecken

```
function klick(i) {  
  if (sperre) return  
  const karte = document.getElementById("k" + i)  
  if (karte.innerHTML !== "?") return // schon offen  
  karte.innerHTML = symbole[i]  
  
  if (erste === -1) {  
    erste = i // erste Karte gemerkt  
  } else {  
    pruefe(i) // zweite -> vergleichen  
  }  
}
```


Paar oder nicht?

```
let zweite = -1

function pruefe(i) {
  zweite = i
  if (symbole[erste] === symbole[zweite]) {
    erste = -1                // Paar gefunden, bleibt offen
    zweite = -1
  } else {
    sperre = true
    setTimeout(klappeZurueck, 800) // gleich wieder zu
  }
}
```

Zurückklappen

Der Timer ruft nach kurzer Zeit `klappeZurueck` auf (wie in Modul 6):

```
function klappeZurueck() {  
  document.getElementById("k" + erste).innerHTML = "?"  
  document.getElementById("k" + zweite).innerHTML = "?"  
  erste = -1  
  zweite = -1  
  sperre = false  
}
```

Übung: dein Memory



PROJEKT

ca. 15 Minuten

- Bring Memory zum Laufen (aufdecken, prüfen, zurückklappen)
- Nimm eigene Symbole (im Browser gehen auch Emojis)

★ **Bonus:** Baue mehr Paare ein, zum Beispiel sechs statt vier.

✓ Das kannst du jetzt

- mit `return` ein Ergebnis aus einer Funktion zurückgeben
- Listen mit `.push` füllen und mit `.includes` durchsuchen
- mit **Listen in Listen** arbeiten
- drei echte Klassiker bauen, die jeder kennt

Du baust jetzt Spiele, die deine Freunde sofort wiedererkennen.

☰ Neue Befehle auf einen Blick

return wert	=	ein Ergebnis zurückgeben
liste.push(x)	=	x hinten anhängen
liste.includes(x)	=	steckt x drin? (true/false)
[[..], [..]]	=	eine Liste aus Listen



Jetzt bist du dran

Du kennst jetzt die Werkzeuge *und* die Klassiker.

Bau deine Lieblingsspiele nach, oder erfinde ganz neue.

Du bist ein echter Spiele-Programmierer geworden.



Du bist jetzt ein Programmierer!

Aus lauter kleinen Bausteinen hast du echte Spiele gebaut.

💡 Tipps für echte Programmierer

- **Ausprobieren:** trau dich, Dinge zu ändern
- **Testen:** ein Stück bauen, dann sofort ausprobieren
- **Fehler sind normal:** auch Profis haben ständig welche
- **Hilfe holen:** fragen ist keine Schwäche

♥ Wie es weitergeht

Du kannst jetzt selbst weitermachen:

- bau deine Projekte weiter aus
- denk dir neue Spiele aus
- zeig deine Seiten deinen Freunden

Danke fürs Mitmachen, und viel Spaß beim Weiterbauen!